Bomblab 实验报告

朱洪儒 25303090027

一、实验环境

• 环境：Linux x86-64，GDB + objdump，课程提供的 bomb 与 defuser.c 模板。

二、通用方法

1. disassemble <func> 或 objdump -d 获取关键函数汇编。
2. 按 SysV AMD64 约定读参：rdi/rsi/rdx/rcx/r8/r9，返回值在 eax。
3. 32 位运算统一写成 uint32_t，利用 C 无符号回绕对应 x86 截断。

三、具体关卡分析

Phase 1

（1）关键汇编：

```
0x00000000004020db <+75>:   call   0x402050 <p1_compute>
0x000000000040206b <+27>:   call   0x402030 <p1_step1>
0x0000000000402076 <+38>:   call   0x402020 <p1_step0>
0x000000000040207f <+47>:   call   0x402040 <p1_step2>
0x0000000000402026 <p1_step0+6>:   lea    0x4a3b(%rdi,%rdi,2),%eax
0x0000000000402034 <p1_step1+4>:   lea    (%rdi,%rdi,2),%eax
0x0000000000402037 <p1_step1+7>:   add    $0xb7e3,%esi
0x0000000000402044 <p1_step2+4>:   lea    (%rdi,%rsi,2),%eax
0x0000000000402047 <p1_step2+7>:   xor    $0xc9d2,%eax
```

（2）恢复出的关系

```
lo  = (uint32_t)challenge;
hi  = (uint32_t)(challenge >> 32);
mid = (uint32_t)(challenge >> 16);
a   = (3u * hi) ^ (mid + 0xb7e3u);
b   = 3u * (uint32_t)(lo - hi) + 0x4a3bu;
ans = (b + 2u * a) ^ 0xc9d2u;
values[0] = ans;
count = 1;
```

（3）推理过程

phase_01 只读 1 个 uint32，比较 x == p1_compute(challenge)。

p1_compute 从 challenge 拆出低 32 位、高 32 位和 challenge >> 16，依次调用三个 step。逐条翻译 lea/sub/xor 即得上式。

Phase 2

（1）关键汇编

```
Value[0]：
   0x0000000000402160 <+112>:   shr    $0x1f,%rax
   0x0000000000402164 <+116>:   add    %eax,%eax
   0x0000000000402166 <+118>:   and    $0x3e,%eax
   0x0000000000402169 <+121>:   add    $0x16,%eax
   0x000000000040216c <+124>:   cmp    %eax,%esi
Value[1] ：（循环 i=1~5）
   0x000000000040217d <+141>:   shr    %cl,%rax
   0x0000000000402180 <+144>:   and    $0x1f,%eax
   0x0000000000402183 <+147>:   xor    %edx,%eax
   0x0000000000402185 <+149>:   lea    (%rax,%rsi,2),%eax
   0x0000000000402188 <+152>:   mov    -0x40(%rbp,%rdx,4),%esi
   0x000000000040218c <+156>:   cmp    %eax,%esi```
```

（2）恢复出的关系：

```
Value[0] = 0x16 + 2 * ((challenge >> 31) & 0x1f);
for (i = 1; i <= 5; i++) {
    shift = 15 + 4 * (i - 1);
    term  = ((challenge >> shift) & 0x1f) ^ i;
    Value[i] = term + 2 * v_{i-1};
}
count = 6;
```

（3）推理过程

第一项由 challenge >> 31 取位后乘 2 加 0x16。循环中 ecx 从 15 每轮加 4，edx 从 1 到 5；and 0x1f 取 5 位，异或轮次 i，再加 2 * v_{i-1}。

Phase 3

（1）关键汇编：

```
   0x0000000000402309 <+89>:    mov    %r12,%rax
   0x000000000040230c <+92>:    mov    %r12,%rsi
```

```
   0x000000000040230f <+95>:    shr    $0x3,%rax
   0x0000000000402313 <+99>:    shr    $0x20,%rsi
   0x0000000000402317 <+103>:   xor    %eax,%esi
   0x0000000000402319 <+105>:   call   0x402250 <p3_dispatch>

   0x000000000040225d <+13>:    notrack jmp *0x405820(,%rax,8)
   0x0000000000402265 <+21>:    nopl   (%rax)
   0x0000000000402268 <+24>:    jmp    0x4021c0 <p3_case_1>
   0x000000000040226d <+29>:    nopl   (%rax)
   0x0000000000402270 <+32>:    jmp    0x4021e0 <p3_case_3>
   0x0000000000402275 <+37>:    nopl   (%rax)
   0x0000000000402278 <+40>:    jmp    0x402210 <p3_case_5>
   0x000000000040227a <+42>:    nopw   0x0(%rax,%rax,1)
   0x0000000000402280 <+48>:    jmp    0x402230 <p3_case_6>
   0x0000000000402282 <+50>:    nopw   0x0(%rax,%rax,1)
   0x0000000000402288 <+56>:    jmp    0x402240 <p3_case_7>
   0x000000000040228a <+58>:    nopw   0x0(%rax,%rax,1)
   0x0000000000402290 <+64>:    jmp    0x4021d0 <p3_case_2>
   0x0000000000402295 <+69>:    nopl   (%rax)
   0x0000000000402298 <+72>:    jmp    0x4021b0 <p3_case_0>
   0x000000000040229d <+77>:    nopl   (%rax)
   0x00000000004022a0 <+80>:    jmp    0x4021f0 <p3_case_4>
   0x00000000004022a5 <+85>:    xor    %eax,%eax
```

x/8a 0x405820 得到：

索引	实际 case
0	p3_case_5
1	p3_case_6
2	p3_case_7
3	p3_case_2
4	p3_case_0
5	p3_case_4
6	p3_case_1
7	p3_case_3

Case0-7：

```
   0x00000000004021b0 <p3_case_0+0>:   endbr64
   0x00000000004021b4 <p3_case_0+4>:   mov    %edi,%eax
   0x00000000004021b6 <p3_case_0+6>:   shr    $1,%edi
   0x00000000004021b8 <p3_case_0+8>:   xor    $0xc6ef,%eax
   0x00000000004021bd <p3_case_0+13>:add    %edi,%eax
   0x00000000004021bf <p3_case_0+15>:  ret
   0x00000000004021c0 <p3_case_1+0>:   endbr64
   0x00000000004021c4 <p3_case_1+4>:   lea
```

```
0x1ff2c(%rdi,%rdi,4),%eax
   0x00000000004021cb <p3_case_1+11>:ret
   0x00000000004021cc:  nopl   0x0(%rax)
   0x00000000004021d0 <p3_case_2+0>:  endbr64
   0x00000000004021d4 <p3_case_2+4>:  lea
0x17c2(%rdi,%rdi,2),%eax
   0x00000000004021db <p3_case_2+11>:ret
   0x00000000004021dc:  nopl   0x0(%rax)
   0x00000000004021e0 <p3_case_3+0>:  endbr64
   0x00000000004021e4 <p3_case_3+4>:  lea    (%rdi,%rdi,1),%eax
   0x00000000004021e7 <p3_case_3+7>:  xor    $0xd63a,%eax
   0x00000000004021ec <p3_case_3+12>:ret
   0x00000000004021ed:  nopl   (%rax)
   0x00000000004021f0 <p3_case_4+0>:  endbr64
   0x00000000004021f4 <p3_case_4+4>:  mov    %edi,%eax
   0x00000000004021f6 <p3_case_4+6>:  shr    $0x2,%edi
   0x00000000004021f9 <p3_case_4+9>:  xor    $0x73ef,%eax
   0x00000000004021fe <p3_case_4+14>: add    %edi,%eax
   0x0000000000402200 <p3_case_4+16>: ret
   0x0000000000402201:  data16 cs nopw 0x0(%rax,%rax,1)
   0x000000000040220c:  nopl   0x0(%rax)
   0x0000000000402210 <p3_case_5+0>:  endbr64
   0x0000000000402214 <p3_case_5+4>:  lea    0x0(,%rdi,4),%eax
   0x000000000040221b <p3_case_5+11>: xor    $0x92dd,%eax
   0x0000000000402220 <p3_case_5+16>: ret
   0x0000000000402221:  data16 cs nopw 0x0(%rax,%rax,1)
   0x000000000040222c:  nopl   0x0(%rax)
   0x0000000000402230 <p3_case_6+0>:  endbr64
   0x0000000000402234 <p3_case_6+4>:  lea
0xb1c3(%rdi,%rdi,2),%eax
   0x000000000040223b <p3_case_6+11>: ret
   0x000000000040223c:  nopl   0x0(%rax)
   0x0000000000402240 <p3_case_7+0>:  endbr64
   0x0000000000402244 <p3_case_7+4>:  lea
0x353b3(%rdi,%rdi,4),%eax
   0x000000000040224b <p3_case_7+11>: ret
```

（2）恢复出的关系

```
first = (uint32_t)challenge & 7u;
arg   = (uint32_t)(challenge >> 32) ^ (uint32_t)(challenge >> 3);
values[0] = first;
values[1] = p3_case_map[first](arg);
```

count = 2;
各 case：
0: (arg ^ 0xc6ef) + (arg >> 1)
1: 5 * arg + 0x1ff2c
2: 3 * arg + 0x17c2
3: (2 * arg) ^ 0xd63a
4: (arg ^ 0x73ef) + (arg >> 2)
5: (4 * arg) ^ 0x92dd
6: 3 * arg + 0xb1c3
7: 5 * arg + 0x353b3

（3）推理过程：

mov %r12d,%edi; and $0x7,%edi 把 challenge 的低 3 位取出来，cmp %edi,-0x18(%rbp) 再与输入的第一个整数比较。因此：v0 = (uint32_t)challenge & 7u;

call p3_dispatch 之前有两条移位和一条异或：
shr $0x3,%rax ; rax = challenge >> 3
shr $0x20,%rsi ; rsi = challenge >> 32
xor %eax,%esi ; esi = 低 32 位(challenge>>32) ^ 低 32 位(challenge>>3)
注意 esi 是 32 位寄存器，所以异或的是两个 32 位截断值：
arg = (uint32_t)(challenge >> 32) ^ (uint32_t)(challenge >> 3);
而 p3_dispatch 开头 mov %edi,%eax; mov %esi,%edi 说明它拿到的第一个参数是 first = challenge & 7，第二个参数是上面的 arg，并把 arg 转给具体 case。因此：
v1 = p3_dispatch(challenge & 7, arg);

★notrack jmp *0x405820(,%rax,8)

这一步是我 7 个函数最后完成的，我第一遍没看见，导致我以为

map[8] = {1, 3, 5, 6, 7, 2, 0, 4};这一步卡了我 2 天

查 map 可知，正确索引是上述基础上把最后两个轮换到前面

所以索引到 case 的映射是：

map[8] = {5, 6, 7, 2, 0, 4, 1, 3};

翻译 8 个 case
每个 case 逐条翻译即可：

```
0: (arg ^ 0xc6ef) + (arg >> 1)
1: 5 * arg + 0x1ff2c
2: 3 * arg + 0x17c2
3: (2 * arg) ^ 0xd63a
4: (arg ^ 0x73ef) + (arg >> 2)
5: (4 * arg) ^ 0x92dd
6: 3 * arg + 0xb1c3
7: 5 * arg + 0x353b3
```

所以：

```
values[0] = (uint32_t)challenge & 7u;
values[1] = p3_case_map[values[0]](arg);
count = 2;
```

Phase 4

（1）关键汇编

1）phase_04 主函数

```
0x00000000004023fb <+91>:   and    $0x7f,%edi
0x00000000004023fe <+94>:   mov    $0x7f,%edx
0x0000000000402403 <+99>:   xor    %eax,%ecx
0x0000000000402405 <+101>:  call   0x402330 <p4_walk>
0x000000000040240a <+106>:  cmp    %eax,-0x14(%rbp)
```

2）p4_walk 基准情况

```
0x000000000040233e <+14>:   cmp    %eax,%edi
0x0000000000402340 <+16>:   je     0x402390 <p4_walk+96>
0x0000000000402390 <+96>:   lea    0xe8(%rdi),%eax
0x0000000000402396 <+102>:  xor    %ecx,%eax
```

3）p4_walk 右分支

```
0x0000000000402352 <+34>:   lea    0x1(%rdi),%esi
0x0000000000402357 <+39>:   call   0x402330 <p4_walk>
0x000000000040235c <+44>:   lea    (%rax,%rax,2),%eax
0x000000000040235f <+47>:   xor    %ebx,%eax
0x0000000000402366 <+54>:   xor    $0x11,%eax
```

4）p4_walk 左分支

```
0x0000000000402370 <+64>:   lea    -0x1(%rdi),%edx
0x0000000000402375 <+69>:   call   0x402330 <p4_walk>
0x000000000040237a <+74>:   lea    (%rax,%rax,2),%eax
0x000000000040237d <+77>:   xor    %ebx,%eax
0x0000000000402384 <+84>:   xor    $0x14,%eax
```

（2）恢复出的关系

```
x = (uint32_t)challenge & 0x7Fu;
c = (uint32_t)(challenge >> 7) ^ (uint32_t)(challenge >> 32);
ans = p4_walk(x, 0u, 127u, c);
values[0] = ans;
count = 1;
```

递归函数定义：

```
uint32_t p4_walk(x, low, high, c) {
    mid = low + (high - low) / 2u;
    if (x == mid)
        return (mid + 0xe8u) ^ c;
    else if (x > mid) {
        res = p4_walk(x, mid + 1u, high, c);
        return (3u * res ^ (mid + c)) ^ 0x11u;
    } else {
        res = p4_walk(x, low, mid - 1u, c);
        return (3u * res ^ (mid + c)) ^ 0x14u;
    }
}
```

（3）推理过程

phase_04 只读 1 个 uint32，比较输入值与 p4_walk（challenge）的返回值是否相等。入口处 challenge 保存在 r12 中：and $0x7f,%edi 取低 7 位作为二分目标值 x；shr $0x7,%rcx 与 shr $0x20,%rax 异或得到递归全程常量 c；初始调用时 esi=0、edx=0x7f，对应二分区间左右边界[0,127]。

p4_walk 为典型二分递归结构：先计算区间中点 mid，若 x 等于 mid 为递归终止基准，返回（mid+0xe8）^c。若 x 大于 mid 则向右半区间递归，子调用返回值先乘 3，再异或 mid + c，最后异或常量 0x11；若 x 小于 mid 则向左半区间递归，运算逻辑相同，末尾异或常量 0x14。最终递归顶层返回值即为口令。

Phase 5

（1）关键汇编

1）phase_05 主函数

```
0x00000000004024d3 <+83>:   call   0x402420 <p5_traverse>
0x00000000004024d8 <+88>:   mov    %eax,%edx
0x00000000004024dc <+92>:   cmp    %edx,-0x1c(%rbp)
0x00000000004024e1 <+97>:   mov    -0x14(%rbp),%eax
0x00000000004024e4 <+100>:  cmp    %eax,-0x18(%rbp)
```

2）p5_traverse 初始化

```
0x0000000000402434 <+20>:    mov    0x405b60(,%rax,8),%rdx
0x0000000000402447 <+39>:    mov    0x8(%rdx),%esi
0x000000000040244c <+44>:    xor    0xc(%rdx),%eax
0x0000000000402451 <+49>:    xor    $0x8016,%eax
0x0000000000402456 <+54>:    mov    %esi,(%r8)
0x0000000000402459 <+57>:    add    %esi,%eax
```

3）p5_traverse 遍历循环

```
0x0000000000402465 <+69>:    mov    (%rdx),%rdx
0x000000000040246b <+75>:    mov    0x8(%rdx),%esi
0x000000000040246e <+78>:    xor    0xc(%rdx),%eax
0x0000000000402471 <+81>:    add    %esi,%eax
0x0000000000402473 <+83>:    mov    %esi,(%r8)
```

（2）恢复出的关系

节点结构：`{ next_ptr(+0), val(+8), xor_val(+12) }`，全局头指针数组 p5_heads[8]地址为 0x405b60。

```
idx   = (uint32_t)challenge & 7u;
steps = ((uint32_t)(challenge >> 3)) & 3u;
cur   = p5_heads[idx];
shift6  = (uint32_t)(challenge >> 6);
high32  = (uint32_t)(challenge >> 32);
acc     = (shift6 ^ high32) ^ cur.xor_val ^ 0x8016u;
acc    += cur.val;
final_val = cur.val;
for (i = 0; i < steps; i++) {
    cur = cur.next;
    acc ^= cur.xor_val;
    acc += cur.val;
    final_val = cur.val;
}
values[0] = acc;
values[1] = final_val;
count = 2;
```

（3）推理过程

phase_05 读取 2 个 uint32，调用 p5_traverse(challenge,&out_val)后执行两次比较：函数返回值与第一个输入比较，指针输出的 out_val 与第二个输入比较。

p5_traverse 中，challenge 低 3 位作为索引，从全局头指针数组 0x405b60 中取出对应链表的起始节点。初始累加值 acc 由 challenge 右移 6 位、高 32

位、节点 xor_val 与常量 0x8016 依次异或，再加上节点 val 得到；同时将首节点 val 写入输出参数。
challenge 右移 3 位后取低 2 位作为遍历步数 steps，范围 0~3。每走一步沿 next 指针移动到下一节点，acc 先异或当前节点 xor_val 再加 val，同时更新输出值为当前节点 val。遍历结束后 acc 作为返回值，口令顺序为 acc 在前、最终节点 val 在后。

Phase 6

（1）关键汇编

1）phase_06 主函数

```
0x0000000000402608 <+72>:    mov    %r12,%rdi
0x000000000040260b <+75>:    call   0x402570 <p6_dispatch>
0x0000000000402610 <+80>:    cmp    %eax,-0x14(%rbp)
```

2）p6_dispatch 两级调用

```
0x0000000000402590 <+32>:    and    $0x38,%ebx
0x0000000000402593 <+35>:    and    $0x7,%eax
0x0000000000402596 <+38>:    call   *0x405e00(,%rax,8)
0x000000000040259d <+45>:    xor    $0x3bd3,%eax
0x00000000004025a2 <+50>:    mov    %eax,%edi
0x00000000004025a4 <+52>:    call   *0x405e00(%rbx)
0x00000000004025af <+63>:    add    $0xdb63,%eax
```

3）核心运算函数

```
p6_op_0: lea 0x4915(%rdi),%eax; shr $1,%edi; xor %edi,%eax
p6_op_1: ror $0x7,%eax
p6_op_2: shr $0x2,%eax; xor %edi,%eax; add $0x7529,%eax
p6_op_3: xor $0xb5a9,%eax; lea (%rax,%rdi,4),%eax
p6_op_4: rol $0x3,%eax; xor $0x76cf,%eax
p6_op_5:    lea    0xf807(,%rdi,4),%eax;    lea    (%rdi,%rdi,2),%edx;
xor %edx,%eax
```

（2）恢复出的关系

```
idx1 = (uint32_t)challenge & 7u;
idx2 = ((uint32_t)challenge >> 3) & 7u;
arg1 = (uint32_t)(challenge >> 6) ^ (uint32_t)(challenge >> 32);
res1 = op_table[idx1](arg1);
arg2 = res1 ^ 0x3bd3u;
res2 = op_table[idx2](arg2);
ans  = res2 + 0xdb63u;
```

```
values[0] = ans;
count = 1;
```

运算函数表（索引 0/2/4 复用 op0）：

```
op0: (x + 0x4915u) ^ (x >> 1)
op1: (x >> 7) | (x << 25)          // 32 位循环右移 7 位
op2: (x ^ (x >> 2)) + 0x7529u
op3: (x ^ 0xb5a9u) + 4u * x
op4: ((x << 3) | (x >> 29)) ^ 0x76cfu  // 32 位循环左移 3 位
op5: (4u * x + 0xf807u) ^ (3u * x)
```

（3）推理过程

phase_06 只读 1 个 uint32，比较输入值与 p6_dispatch（challenge）的返回值是否相等。p6_dispatch 基于同一张函数指针表 0x405e00 完成两级间接调用。
第一级：challenge 低 3 位为索引 idx1，参数 arg1 由 challenge 右移 6 位与高 32 位异或得到，调用表中对应运算函数得 res1。
第二级：res1 异或常量 0x3bd3 得到 arg2；challenge 第 3～5 位为索引 idx2（对应地址偏移&0x38），调用同表中对应函数得 res2。
最终返回值为 res2 加常量 0xdb63，即为口令。函数表中索引 0、2、4 复用同一函数 op0，其余索引各自对应独立运算函数；其中 ror/rol 为 32 位循环移位，还原时需通过高低位拼接实现等价运算。

7. Secret Phase（Honor 关）

（1）关键汇编

1）secret_gate 入口格式校验

```
0x000000000040274c <+60>:   call   0x401190 <strlen@plt>
0x0000000000402751 <+65>:   sub    $0x24,%rax
0x0000000000402755 <+69>:   cmp    $0x7f,%rax
0x000000000040277b <+107>:  cmpl   $0x4e45504f,-0xb0(%rbp)
```

2）secret_mix 核心运算

```
0x0000000000402626 <+6>: ror    $1,%eax
0x0000000000402628 <+8>: xor    $0xb337,%eax
```

3）secret_phase 三级校验逻辑

```
0x0000000000402695 <+101>:  and    $0x3,%ecx
0x0000000000402698 <+104>:  cmp    %ecx,-0x2c(%rbp)
0x00000000004026a0 <+112>:  imul   $0x1553,%ecx,%r9d
0x00000000004026b2 <+130>:  xor    %r12d,%esi
0x00000000004026b5 <+133>:  xor    %r13d,%r8d
```

```
0x00000000004026c4 <+148>:  call   0x402620 <secret_mix>
0x00000000004026c9 <+153>:  xor    %r9d,%eax
0x00000000004026cc <+156>:  cmp    %eax,-0x28(%rbp)
0x00000000004026d1 <+161>:  shr    $1,%esi
0x00000000004026d3 <+163>:  lea    0xa664(%r8,%rcx,1),%edx
0x00000000004026db <+171>:  add    %esi,%edx
0x00000000004026dd <+173>:  xor    %edx,%eax
0x00000000004026df <+175>:  cmp    -0x24(%rbp),%eax
```

（2）恢复出的关系

输入格式：字符串前 32 字节为任意填充，紧接着固定标记 OPEN，之后为三个空格分隔的无符号十进制整数；总长度满足总长度-36≤127。

```
c = (uint32_t)context & 3u;
esi = (uint32_t)(challenge >> 32) ^ (uint32_t)challenge;
r8d = (uint32_t)(context >> 32) ^ (uint32_t)context;
edi = esi ^ r8d;
r9d = c * UINT32_C(0x1553);
r9d ^= (uint32_t)(context >> 16);
```

辅助函数 secret_mix：

```
uint32_t secret_mix(uint32_t x) {
    x = (x>>1) | (x<<31);   //32 位循环右移 1 位
    return x ^ UINT32_C(0xb337);
}
```

最终三个口令整数：

```
v0 = c;
v1 = secret_mix(edi) ^ r9d;
v2 = v1 ^ (UINT32_C(0xa664) + r8d + c + (esi >> 1));
count = 3;
```

（3）推理过程

1）输入格式还原：从 secret_gate 入手，先调用 strlen 计算输入总长度，减去常量 0x24（36）后与 0x7f 比较，说明有效数字部分长度不超过 127 字节，总长度至少为 36 字节。随后栈底偏移处检查四字节常量 0x4e45504f，小端序对应 ASCII 字符串 OPEN，说明输入包含固定标记。结合传入`secret_phase`的指针偏移（标记后 4 字节开始解析整数），可还原输入结构为「32 字节填充 + 固定字符串 OPEN + 三个整数字符串」，填充 32 字节时总长度恰好满足最小 36 字节的要求。

2）第一个整数推导：secret_phase 中将 context 低 2 位与 0x3 按位与，再与第一个输入整数比较，因此第一个口令值固定为 context &3。

3）中间变量推导：challenge 的高 32 位与低 32 位异或得到 esi；context 的高 32 位与低 32 位异或得到 r8d；二者再次异或得到 edi，作为 secret_mix 的入参。context 低 2 位乘以常量 0x1553，再异或 context 右移 16 位的值，得

到 r9d。
4）第二个整数推导：secret_mix 对输入执行 32 位循环右移 1 位，再异或常量 0xb337 返回；返回值与 r9d 异或，结果与第二个输入整数相等。
5）第三个整数推导：以常量 0xa664 为基底，叠加 r8d、第一个整数 c，以及右移 1 位后的 esi，得到中间值 edx；第二个整数的期望值与 edx 异或，结果与第三个输入整数相等。

四、运行结果与截图

每个已完成关卡至少一次使用公开列表以外的 seed 验证的记录：



第 1 关 · 维护站
终端上留着一份维护记录。
你按了一下回车，屏幕亮起，六个指示灯中只有第一个在闪。
林夏把旁边的椅子拖过来：‘我看记录，你看程序。’
终端能读到接入请求，但还没有放行。旧的口令工具已经找不到了，
留下来的程序仍然在等一个合法的输入。

Phase 1 challenge: 0xbfc846100bfc1e42
Input (unsigned decimal integers):
Phase 1 defused.

门锁响了一声，维护站开始接收新的记录。
日志里的请求时间各不相同，城市的日期却始终是同一天。
其中一份校时记录来自街那头的钟楼。林夏记下地址，推开门。

[1 已解除] [2 当前] [3 待解除] [4 待解除] [5 待解除] [6 待解除]

第 2 关 · 钟楼
钟楼没有钟声。齿轮照常转动，日期却停在昨天。
林夏翻开底座里的记录：同一天的几次校时被写在不同位置，
每一次都接着上一次的结果。你们得先确认这串记录能接起来。
窗外的送货车又绕过街角。司机还是刚才那句话：
‘今天来不及了。明天一定送到。’

Phase 2 challenge: 0x987bbcbfdd7e532f
Input (unsigned decimal integers):
Phase 2 defused.

校时检查通过了。钟楼送出一条新记录，几秒后又收回原样的一条。
‘不是钟没走。’林夏把两条记录放在一起，‘它送出去又回来了。’
你们沿着线路往下查，重复的地址最后指向街口的分流站。

[1 已解除] [2 已解除] [3 当前] [4 待解除] [5 待解除] [6 待解除]

第 3 关 · 分流站
沿着钟楼的线路，你们来到街口的分流站。
屏幕上的请求到达一个路口，又被送回前一个路口。
林夏在纸上划掉了一条重复路线：‘它一直在旧地址之间绕。’
站里保留着几种不同的处理办法。请求的标记决定它会走哪一条。

Phase 3 challenge: 0xc3f2827affe7f664
Input (unsigned decimal integers):
Phase 3 defused.

这次请求没有再绕回来。分流站把它送进档案馆，收到一条回复：
‘未找到交接存档。’回复后面附着一份旧目录。
林夏收好路线图。你们穿过马路，档案馆的门刚刚打开。

[1 已解除] [2 已解除] [3 已解除] [4 当前] [5 待解除] [6 待解除]

第 4 关 · 档案馆
档案馆里没有人，书架上的指示灯一层层向深处延伸。
原来的存档经过几次搬迁，被分在不同的目录里。
你们找到入口，后面的路还要沿着旧索引逐层查下去。
林夏把刚才的纸翻到背面：‘这次别凭房间号猜，跟着记录走。’

Phase 4 challenge: 0x4fc446b53f17fb29
Input (unsigned decimal integers):
Phase 4 defused.

最后一层索引返回了一份完整存档，还有那次迁移留下的记录。
新设备接上时，交接没有做完。城市只能反复恢复最后一份旧存档；
居民今天做过的事，到了夜里就都没了。
存档后面列着几处还没核对的设备。你认出了维修铺那盏灯。

你们走回街上。送货司机已经发动了车：
‘昨天耽误了。今天得早点走。’
维修铺的灯装好了，老板正把那张写着‘明天装上’的纸条揭下来。
林夏站在橱窗前看了一会儿，跟上了你。

Honor · 观测台旧备份

林夏准备关闭档案列表，忽然指着最下面一项：‘这里还有个备份。’
文件夹叫‘留给后来的人’，署名是 sky 和许遥。
扉页写着：‘交接完成后，如果你还有时间，可以到观测台坐坐。’
这份旧档案不参与城市的运行。你们沿着楼梯登上观测台，
桌上的阅读器仍在等待旧程序认可的输入。
城市已经安全了。是否继续读下去，由你们自己决定。

Secret phase defused.
[SAFE] 城市照常运行。附加档案已恢复。

阅读器亮起来。最先出现的不是代码，是两份很早的留言。

  许遥：

  北方那边的通知到了，我准备去。
  那个项目我想试很久了。你帮我看申请的时候，我还说不一定选得上，
  现在真能去了，反而有点不习惯。以后遇到问题，
  不能再把椅子滑到你旁边，问一句‘你看这是不是有毛病’。

  你想留下继续手里的事，我记得你为它准备了多久。
  我也想看看，最后能做成什么样。
  这里没做完的几处，我把记录整理好了。
  你有空就改，没空先忙自己的事。

  走之前出去吃顿饭吧。这次别在实验室凑合了。

  —— sky

隔了几天，许遥在后面添了一份回复。

  吃饭的时间你定。

第 5 关 · 巡检回廊
巡检回廊连着档案馆和街区。墙上的设备按编号排得很整齐，
线缆却从一台背后穿过去，接到另一排的某个接口。
维修铺的老板把一盏没装好的灯放到窗边。
底座上贴着纸条：‘今天做好接口，明天装上。’
林夏认出那盏灯：‘我们刚进来时，他就在做这个。’

Phase 5 challenge: 0x58bc3cb37bc7b2b3
Input (unsigned decimal integers):
Phase 5 defused.

最后一处连接也对上了，今天的设备状态第一次进入待保存的记录。
维修铺老板试着拧了一下灯座，灯亮了一瞬，又被他小心地关掉。
‘留到明天装好。’他说。
交接清单上只剩最后一项。你们带着记录走到回廊尽头。

[1 已解除] [2 已解除] [3 已解除] [4 已解除] [5 已解除] [6 当前]

第 6 关 · 交接室
交接室的灯还亮着。桌上留着一份没有签完的记录。
你们刚才找回的时间、路线、存档和设备状态都到了这里。
林夏把记录摊开，逐项核对。你接上最后一台终端。
它会根据当前状态选择处理例程，再检查最后的交接结果。
走廊里传来送货车的声音。你们把门掩上，继续看屏幕。

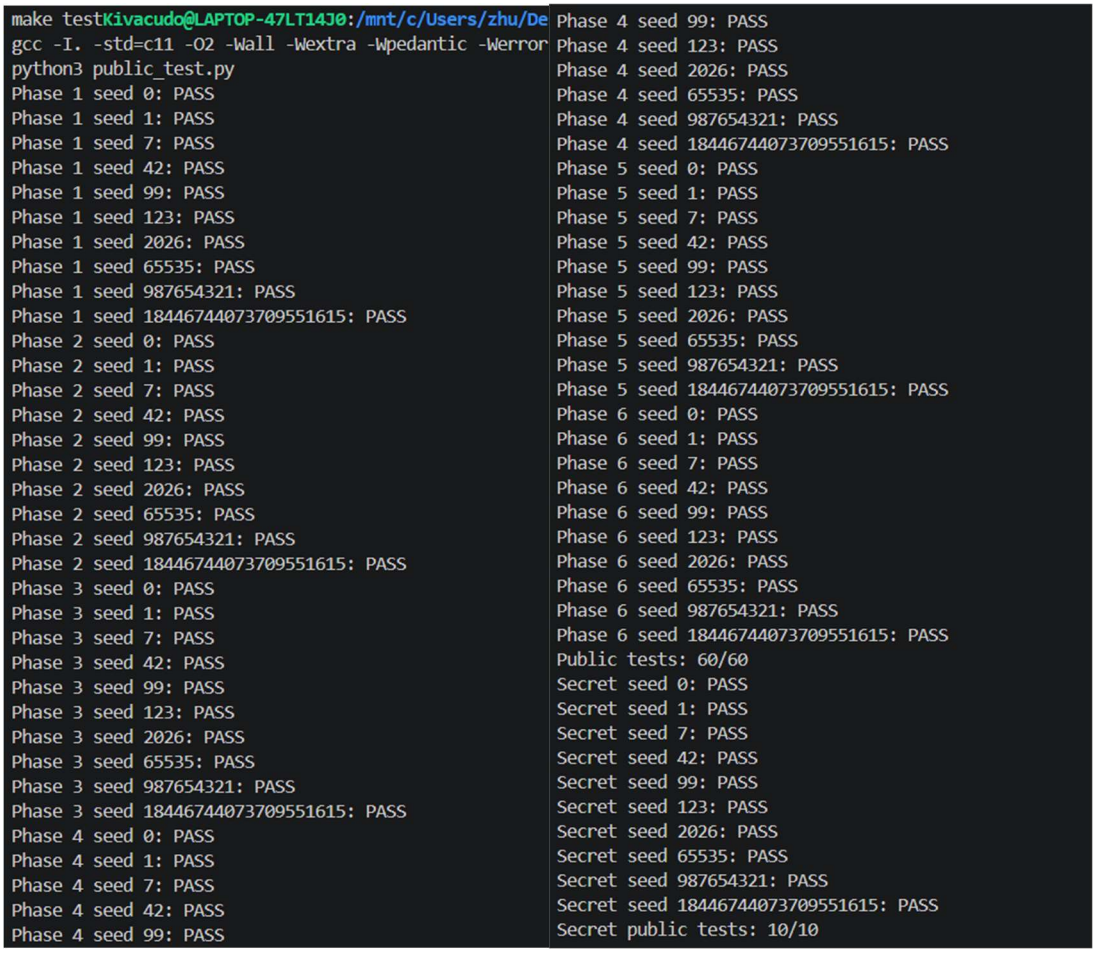
Phase 6 challenge: 0xb9f24f7bae4a6586
Input (unsigned decimal integers):
Phase 6 defused.

终端逐项读过记录，最后停在‘保存完成’。
林夏没有马上起身。你们等了一会儿，屏幕上的日期向前走了一天。
这次没有恢复旧存档。

[1 已解除] [2 已解除] [3 已解除] [4 已解除] [5 已解除] [6 已解除]

Congratulations! You've defused the bomb.
[SAFE] 六项检查全部通过。新存档已启用。

已完成关卡的运行结果截图：



五、引用资料与辅助工具

引用资料

课程 Bomblab 讲义、defuser.c 模板与驱动程序接口说明。
GDB 文档：disassemble、x、break、p/x。
GNU Binutils objdump 文档。
Intel SDM：移位、lea、跳转表。
System V AMD64 ABI：整数参数寄存器与返回值。
C11 标准：无符号回绕、snprintf 截断检查。

辅助工具用途

GDB：查看汇编、查看跳转表、断点比较寄存器、检查栈上输入值。
objdump：批量查看汇编与确认函数边界。
Github:提交平台，顺带提供暂存等服务